

Oracle SQL – Funzioni Avanzate e Logica nella SELECT

Una lezione pratica per imparare a usare funzioni avanzate, inserire logica condizionale nelle query e preparare il terreno per PL/SQL. Durata: ~2 ore. Dataset di riferimento: employees e departments.

LIVELLO: BASE / INTERMEDIO

ORACLE DATABASE

2 ORE

Obiettivi della Lezione

In questa sessione impareremo a trasformare i dati direttamente in SQL, senza ricorrere a elaborazioni esterne. Le competenze acquisite oggi sono fondamentali per qualsiasi sviluppatore o analista che lavori con Oracle Database in ambienti professionali.

Funzioni Avanzate

Usare funzioni di stringa, numeriche e di data per trasformare i dati direttamente nella SELECT

Logica nelle Query

Inserire logica condizionale con CASE e DECODE per classificare e categorizzare i risultati

Verso PL/SQL

Preparare le basi concettuali per il passaggio alla programmazione PL/SQL avanzata

 Prerequisiti: SELECT base, WHERE, JOIN, GROUP BY. Dataset: schema HR con tabelle `employees` e `departments`.

Logica Condizionale con CASE

L'istruzione **CASE** permette di inserire logica condizionale direttamente in una query SQL. È uno degli strumenti più potenti per classificare, categorizzare e trasformare i dati al volo. Oracle supporta due forme di CASE: il **CASE semplice** (confronta un'espressione con valori fissi) e il **CASE cercato** (valuta condizioni booleane arbitrarie con operatori di confronto). La seconda forma è più flessibile ed è quella utilizzata nella maggior parte degli scenari reali.

Sintassi Base

```
CASE
  WHEN condizione1 THEN risultato1
  WHEN condizione2 THEN risultato2
  ELSE risultato_default
END
```

Il CASE può essere usato nella SELECT, nell'ORDER BY, nel GROUP BY e persino nella clausola WHERE per filtrare in modo dinamico.

Esempio Pratico – Classificazione Stipendio

```
SELECT first_name,
       salary,
       CASE
         WHEN salary < 3000 THEN 'LOW'
         WHEN salary < 6000 THEN 'MEDIUM'
         ELSE 'HIGH'
       END AS salary_level
FROM employees;
```

Questo esempio classifica ogni dipendente in base al livello di stipendio. Notare come il CASE restituisca una stringa testuale derivata da una condizione numerica.

CASE Avanzato – Colonne Multiple e ORDER BY

Il CASE può essere utilizzato su più colonne contemporaneamente e può anche controllare l'ordinamento dei risultati in modo dinamico. Questo è particolarmente utile per creare report ordinati secondo criteri di business personalizzati, non semplicemente alfabetici o numerici. Ad esempio, è possibile definire priorità di reparto, livelli di urgenza o categorie di prodotto direttamente nell'ORDER BY, senza bisogno di colonne aggiuntive nella tabella.

CASE su Più Colonne

```
SELECT first_name,  
       department_id,  
       CASE  
         WHEN department_id = 50 THEN 'IT'  
         WHEN department_id = 60 THEN 'SALES'  
         ELSE 'ALTRO'  
       END AS dept_name,  
       CASE  
         WHEN salary > 8000 THEN 'SENIOR'  
         ELSE 'JUNIOR'  
       END AS livello  
FROM employees;
```

In questo esempio due CASE indipendenti trasformano rispettivamente l'ID del dipartimento in nome leggibile e lo stipendio in livello gerarchico. Ogni CASE valuta una colonna diversa e restituisce un alias distinto.

CASE in ORDER BY

```
SELECT first_name, department_id  
FROM employees  
ORDER BY  
  CASE  
    WHEN department_id = 50 THEN 1  
    WHEN department_id = 60 THEN 2  
    ELSE 3  
  END;
```

L'uso di CASE nell'ORDER BY consente di definire un ordinamento personalizzato che non corrisponde all'ordine alfabetico o numerico naturale. I dipartimenti IT (50) e Sales (60) vengono mostrati per primi, tutti gli altri dopo.



💡 Il CASE in ORDER BY è ideale per ordinare per priorità di business: ad esempio, mostrare prima i clienti VIP, poi i clienti standard, infine i potenziali.

Gestione dei NULL – NVL e COALESCE

In Oracle, NULL rappresenta l'assenza di un valore. Non è zero, non è una stringa vuota: è "nessun valore". Questo comportamento può causare risultati inattesi nelle query, specialmente nelle operazioni aritmetiche e nelle concatenazioni. Le funzioni NVL e COALESCE permettono di sostituire i NULL con valori di default significativi.

NVL – Sintassi Oracle

```
SELECT first_name,  
  
NVL(commission_pct, 0)  
AS comm  
FROM employees;
```

NVL(colonna, valore_default)
restituisce il valore della colonna
se non è NULL, altrimenti
restituisce il valore di default.
Funzione specifica di Oracle.

COALESCE – Standard ANSI

```
SELECT first_name,  
  
COALESCE(commission_pct,  
          salary *  
          0.1,  
          0) AS  
comm_stimata  
FROM employees;
```

COALESCE(val1, val2, val3,
...) restituisce il primo valore
non NULL trovato. È portabile su
tutti i database e supporta più
alternative.

NULL nelle Operazioni

```
-- Attenzione: NULL +  
numero = NULL  
SELECT salary +  
commission_pct  
FROM employees;  
  
-- Soluzione corretta  
SELECT salary +  
NVL(commission_pct, 0)  
FROM employees;
```

Senza NVL, qualsiasi operazione
aritmetica che coinvolge NULL
restituisce NULL. Questo è uno
degli errori più comuni nelle
query di calcolo.

Funzioni Stringa Avanzate

Oracle offre un ricco set di funzioni per manipolare le stringhe direttamente in SQL. Queste funzioni sono essenziali per pulire, formattare e trasformare dati testuali prima della visualizzazione o dell'esportazione. Le quattro funzioni principali — `REPLACE`, `SUBSTR`, `INSTR` e `LENGTH` — coprono la maggior parte delle esigenze di manipolazione testuale.



REPLACE

Sostituisce tutte le occorrenze di una sottostringa con un'altra.

```
REPLACE(first_name, 'a', 'X')
```



INSTR

Restituisce la posizione di una sottostringa all'interno di una stringa.

```
INSTR(email, '@', 1)
```



SUBSTR

Estrae una porzione di stringa a partire da una posizione.

```
SUBSTR(first_name, 1, 3)
```



LENGTH

Restituisce la lunghezza in caratteri di una stringa.

```
LENGTH(first_name)
```

Esempio Combinato

```
SELECT first_name,  
       REPLACE(first_name, 'a', 'X') AS nome_modificato,  
       SUBSTR(first_name, 1, 3) AS prefisso,  
       INSTR(first_name, 'a') AS pos_a,  
       LENGTH(first_name) AS lungh  
FROM employees;
```

Questo esempio dimostra come le quattro funzioni possano essere usate insieme nella stessa `SELECT` per analizzare e trasformare contemporaneamente i dati testuali.

Concatenazione Avanzata e Costruzione Stringhe

La concatenazione di stringhe in Oracle si esegue con l'operatore `||`. È uno strumento fondamentale per costruire nomi completi, codici identificativi, messaggi e chiavi composite direttamente nella query. La concatenazione può combinare colonne, letterali e risultati di funzioni in un'unica espressione.

Operatore `||`

```
SELECT first_name || ' ' || last_name
       AS nome_completo
FROM employees;

-- Con alias di colonna
SELECT employee_id,
       'EMP-' || employee_id AS codice
FROM employees;
```

L'operatore `||` concatena due espressioni. Per concatenare più elementi, si usa in cascata. Gli spazi e i separatori vanno inseriti come letterali tra apici singoli.

Costruzione Stringhe Complesse

```
SELECT
    first_name || ' ' || last_name
    || ' (' || job_id || ')'
    || ' - Dept: ' || department_id
    AS scheda_dipendente
FROM employees;
```

Combinando `||` con letterali testuali si possono costruire stringhe descrittive complete, utili per report, export CSV o interfacce utente.



⚠️ Attenzione: se una delle colonne concatenate è NULL, il risultato dell'intera concatenazione sarà NULL. Usare sempre `NVL` sulle colonne opzionali.

Funzioni Numeriche e Date

Oracle fornisce funzioni dedicate per arrotondare numeri, troncare date e calcolare differenze temporali. Queste funzioni sono essenziali per report finanziari, analisi di performance e calcolo di anzianità o scadenze. La gestione delle date in Oracle è particolarmente potente grazie alla possibilità di eseguire operazioni aritmetiche dirette.

ROUND e TRUNC

```
SELECT
  ROUND(salary, -2) AS
  salary_arrotondata,
  TRUNC(salary, -2) AS
  salary_troncata
FROM employees;
```

ROUND arrotonda al valore più vicino; **TRUNC** tronca senza arrotondare. Il secondo parametro indica la precisione (positivo per decimali, negativo per cifre intere).

SYSDATE

```
SELECT SYSDATE AS oggi,
       TRUNC(SYSDATE) AS
  oggi_senza_ora
FROM dual;
```

SYSDATE restituisce data e ora correnti del server Oracle.

TRUNC(SYSDATE) rimuove la parte oraria, restituendo solo la data a mezzanotte.

Operazioni su Date

```
SELECT first_name,
  hire_date,
         SYSDATE -
  hire_date AS
  giorni_lavorati,
         ROUND((SYSDATE -
  hire_date)/365, 1)
         AS anni_lavorati
FROM employees;
```

In Oracle, sottrarre due date restituisce il numero di giorni. Dividendo per 365 si ottengono gli anni approssimativi. Utile per calcolare anzianità e scadenze.

DECODE – L'Alternativa Oracle a CASE

DECODE è una funzione specifica di Oracle che offre una sintassi più compatta rispetto al CASE per confronti di uguaglianza. È supportata da decenni in Oracle e ancora ampiamente utilizzata in codice legacy. Tuttavia, per condizioni complesse o non basate su uguaglianza, il CASE rimane la scelta preferibile per leggibilità e standard SQL.

Sintassi DECODE

```
DECODE(espressione,  
       valore1, risultato1,  
       valore2, risultato2,  
       valore_default)
```

Esempio Pratico

```
SELECT first_name,  
       DECODE(department_id,  
              50, 'IT',  
              20, 'HR',  
              60, 'SALES',  
              'ALTRO') AS dept_nome  
FROM employees;
```

DECODE confronta `department_id` con ogni valore e restituisce il risultato corrispondente. Se nessun valore corrisponde, restituisce il default.

DECODE vs CASE

DECODE

- Sintassi più compatta
- Solo confronti di uguaglianza
- Specifico di Oracle
- Legacy, meno leggibile

CASE

- Standard SQL ANSI
- Condizioni arbitrarie (>, <, BETWEEN)
- Più leggibile e mantenibile
- Portabile su tutti i DB



✓ Consiglio: preferire sempre CASE per nuovo codice. Usare DECODE solo per compatibilità con codice esistente.

Combinare Funzioni – Esercitazioni e Riepilogo

La vera potenza di SQL emerge quando si combinano più funzioni in un'unica espressione. Logica condizionale, trasformazione stringhe e gestione dei NULL possono lavorare insieme per produrre output complessi direttamente dalla query. Le esercitazioni seguenti consolidano tutti i concetti della lezione.

1

Esercizio 1 – CASE

Classifica i dipendenti in tre fasce salariali (LOW, MEDIUM, HIGH) e ordina per fascia.

```
SELECT first_name, salary,
       CASE WHEN salary < 3000 THEN 'LOW'
            WHEN salary < 6000 THEN 'MEDIUM'
            ELSE 'HIGH'
       END AS fascia
FROM employees
ORDER BY fascia;
```

2

Esercizio 2 – NVL

Calcola il reddito totale (stipendio + commissione) gestendo i NULL.

```
SELECT first_name,
       salary + NVL(salary * commission_pct, 0)
       AS reddito_totale
FROM employees;
```

3

Esercizio 3 – Stringhe

Costruisci un codice utente con prime 3 lettere del nome in maiuscolo.

```
SELECT first_name,
       UPPER(SUBSTR(first_name, 1, 3))
       || '_USER' AS user_code
FROM employees;
```

4

Esercizio 4 – Combinato

Unisci CASE, NVL e funzioni stringa in una query di report completa.

```
SELECT
  UPPER(SUBSTR(first_name,1,1))
  || '. ' || last_name AS nome_fmt,
  NVL(TO_CHAR(commission_pct),'N/A')
  AS comm,
  CASE WHEN SYSDATE-hire_date>3650
        THEN 'SENIOR' ELSE 'JUNIOR'
  END AS anzianita
FROM employees;
```

Concetti Chiave della Lezione

☒ Logica in SQL

CASE e DECODE inseriscono logica condizionale direttamente nelle

🔄 Trasformazione Dati

Funzioni stringa, numeriche e date trasformano i dati al volo

🔗 Verso PL/SQL

Questi concetti sono la base per blocchi PL/SQL, stored procedure e